

Entry point

```
import { createRoot } from 'react-dom/client'
```

index.html

```
<div id="root"></div>  
<script type="module" src="/src/main.jsx"></script>
```

```
createRoot(document.getElementById('root')).render(<App />)
```

```
import './App.css'
```

```
function App() {  
  return (  
    <div>  
      <h1>Welcome to React</h1>  
      <p>This is a sample React application.</p>  
    </div>  
  )  
}
```

```
export default App
```

XML

npm run dev
package.json

"dev": "vite",

<Filter />

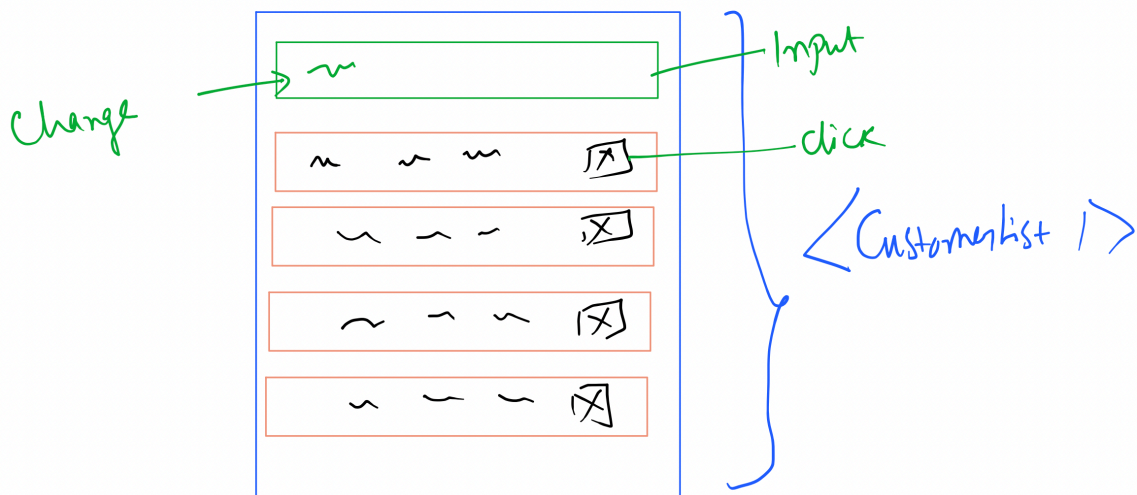
<CustomerRow />

<CustomerRow />

<CustomerRow />

<CustomerRow />

<Customerlist />



PROPS

`<CustomerRow customer={c} />`

PROPS

```

this.state.customers.map(c => <CustomerRow
  [ delEvt = {(id) => this.deleteCustomer(id)}
    customer={c} /> ]
  )

deleteCustomer(id) {
  let custs = this.state.customers.filter(c => c.id !== id);
  // avoid below statement
  // this.state.customers = custs; // state is modified, but
  // below code not only updates state but
  // also triggers reconciliation
  this.setState({
    customers: custs
  });
}

export default class CustomerRow extends Component {
  deleteRow(id) {
    console.log("Delete <CustomerRow /> ", id);
    this.props.delEvt(id);
  }

  render() {
    // destructuring
    let {id, firstName, lastName, gender, imageUrl} = this.props.customer
    return <div>
      <img src={imageUrl} />
      <br/>
      {firstName} {lastName}
      &nbsp;
      <button type="button" onClick={() => this.deleteRow(id)}>Delete</button>
    </div>
  }
}

```

Diagram illustrating the flow of props and state management. A green arrow labeled "2" points from the `delEvt` prop in the `<CustomerRow />` call to the `deleteRow` method in the `CustomerRow` class. Another green arrow points from the `deleteRow` method to the `deleteCustomer` method in the parent component. A third green arrow points from the `customer` prop in the `<CustomerRow />` call to the destructuring assignment in the `render` method of the `CustomerRow` class.